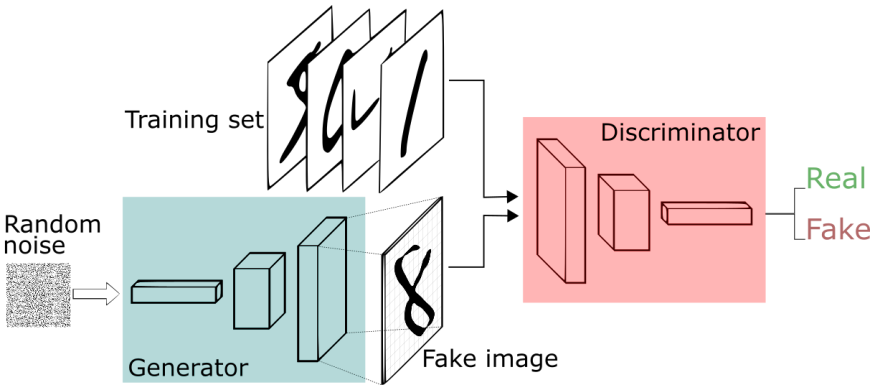
ELO328 - Procesamiento Digital de Imágenes

Clase 13: Redes generativas adversativas

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

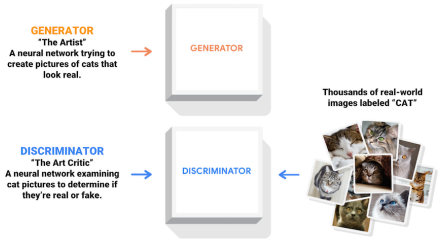
Generative Adversarial Networks (GANs)



Introducción a las GANs

Dos modelos son entrenados simultáneamente por un proceso contradictorio:

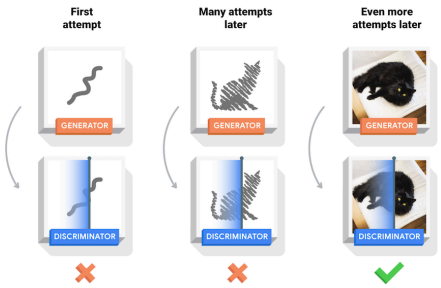
- Un *generador* ("el artista") aprende a crear imágenes que parecen reales.
- Un *discriminador* ("el crítico de arte") aprende a diferenciar las imágenes reales de las falsificaciones.



Proceso de entrenamiento

- Durante el entrenamiento, el *generador* mejora progresivamente en la creación de imágenes que parecen reales, mientras que el *discriminador* mejora en diferenciarlas.

- El proceso alcanza el equilibrio cuando el *discriminador* ya no puede distinguir las imágenes reales de las falsificaciones.



Ejemplo GAM con el Fashion MNIST dataset

In [30]:
```
import tensorflow as tf
```

In [31]:
```
# Cargar los datos del Fashion-MNIST dataset
# No necesitamos etiquetas (aprendizaje no supervisado)
(train_images, _), (_, _) = tf.keras.datasets.fashion_mnist.load_data()
```

In [32]:
```
# Normalizar entre (0, 1)
X_train = train_images.astype("float32") / 255
print(X_train.shape)
```

```
(60000, 28, 28)
```

Generador

In [33]:
```
codings_size = 30

generator = keras.models.Sequential([
    keras.layers.Dense(100, activation="selu", input_shape=[codings_size]),
    keras.layers.Dense(150, activation="selu"),
    keras.layers.Dense(28 * 28, activation="sigmoid"),
    keras.layers.Reshape([28, 28])
])
```

Discriminador

In [34]:
```
discriminator = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(150, activation="selu"),
    keras.layers.Dense(100, activation="selu"),
    keras.layers.Dense(1, activation="sigmoid")
])
```

GAN

In [35]:
```
gan = keras.models.Sequential([generator, discriminator])
```

Entrenamiento

In [36]:
```
def plot_multiple_images(images, n_cols=None):
    n_cols = n_cols or len(images)
    n_rows = (len(images) - 1) // n_cols + 1
    if images.shape[-1] == 1:
        images = np.squeeze(images, axis=-1)
    plt.figure(figsize=(n_cols, n_rows))
    for index, image in enumerate(images):
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(image, cmap="binary")
        plt.axis("off")
```

In [37]:
```
def train_gan(gan, dataset, batch_size, codings_size, n_epochs=50):
    generator, discriminator = gan.layers
    for epoch in range(n_epochs):
        print(f"Epoch [{epoch+1}/{n_epochs}] ...")
        for x_batch in dataset:
            # Fase 1 - entrenamiento del generador
            noise = tf.random.normal(shape=(batch_size, codings_size))
            generated_images = generator(noise)
            # Reunir igual numero de imagenes generadas (clase 0) y reales (clas
            x_discr = tf.concat([generated_images, x_batch], axis=0)
            y_discr = tf.constant([[0.0]] * batch_size + [[1.0]] * batch_size)
            discriminator.train_on_batch(x_discr, y_discr)
            # --
            # Fase 2 - entrenamiento del discriminador
            noise = tf.random.normal(shape=(batch_size, codings_size))
            # Las imagenes generadas deben ser etiquetadas como "reales" por el
            y_gen = tf.constant([[1.0]] * batch_size)
            # Actualizar los pesos del generador
            gan.train_on_batch(noise, y_gen)
        plot_multiple_images(generated_images, 8)
        plt.show()
```

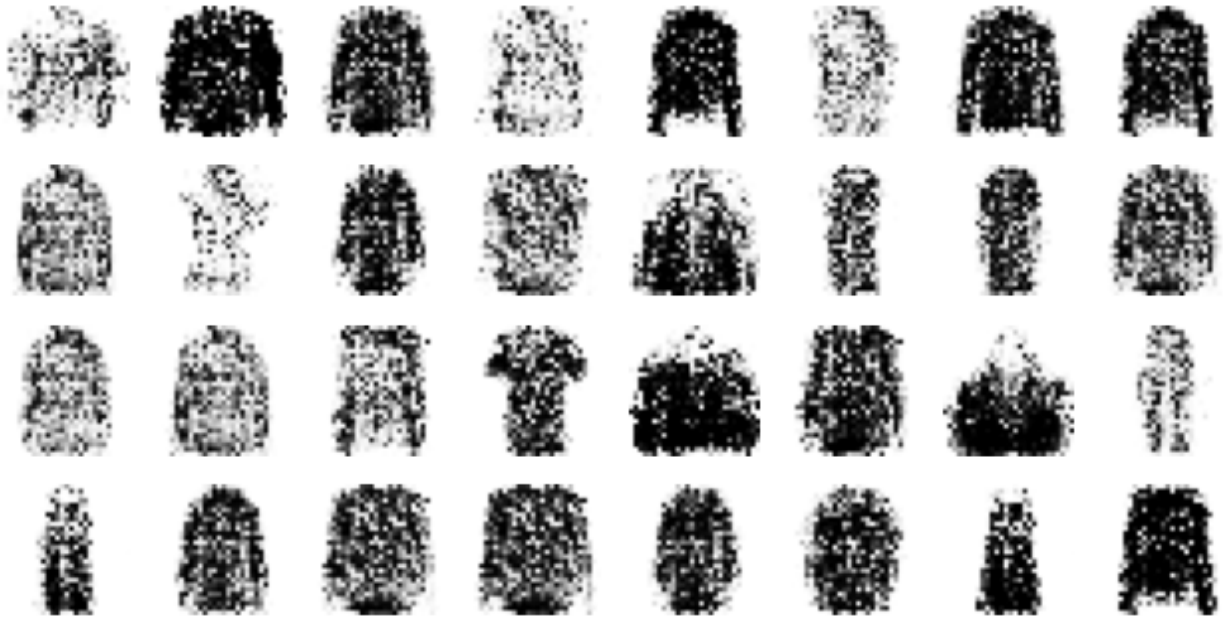
In [38]:
```
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
discriminator.trainable = False
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

In [39]:
```
# Cargar las imagenes en batches
batch_size = 32
dataset = tf.data.Dataset.from_tensor_slices(X_train).shuffle(1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

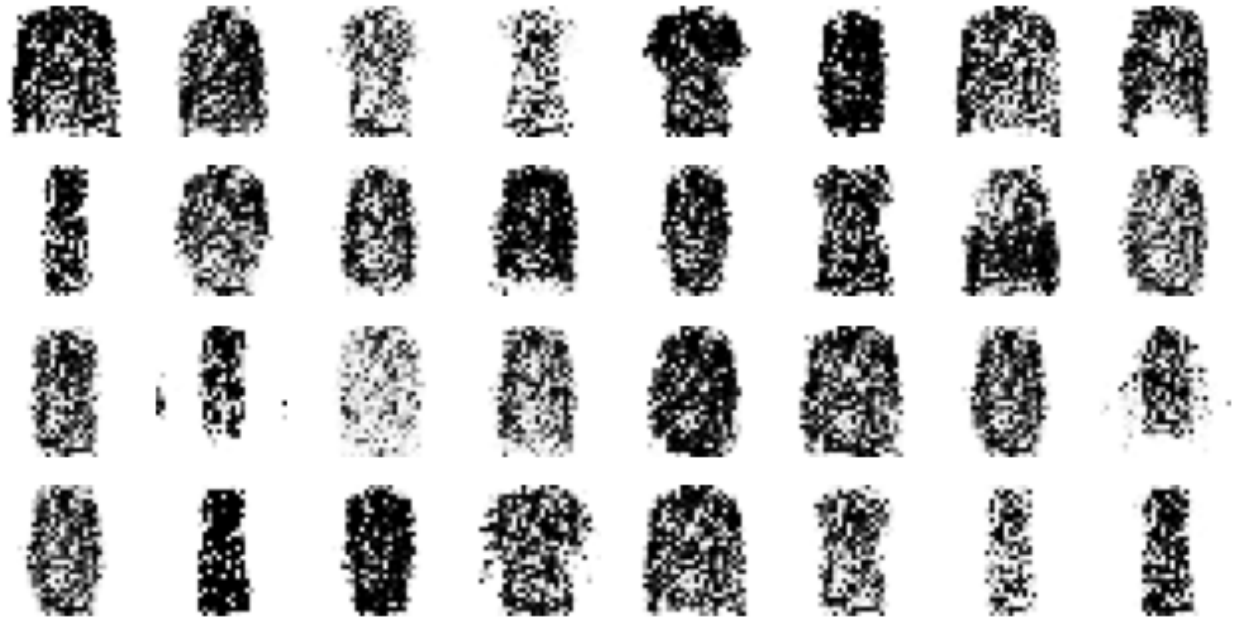
In [40]:

```
# Entrenar el modelo GAN con 5 iteraciones
train_gan(gan, dataset, batch_size, codings_size, n_epochs=5)
```

```
Epoch [1/5] ...
```



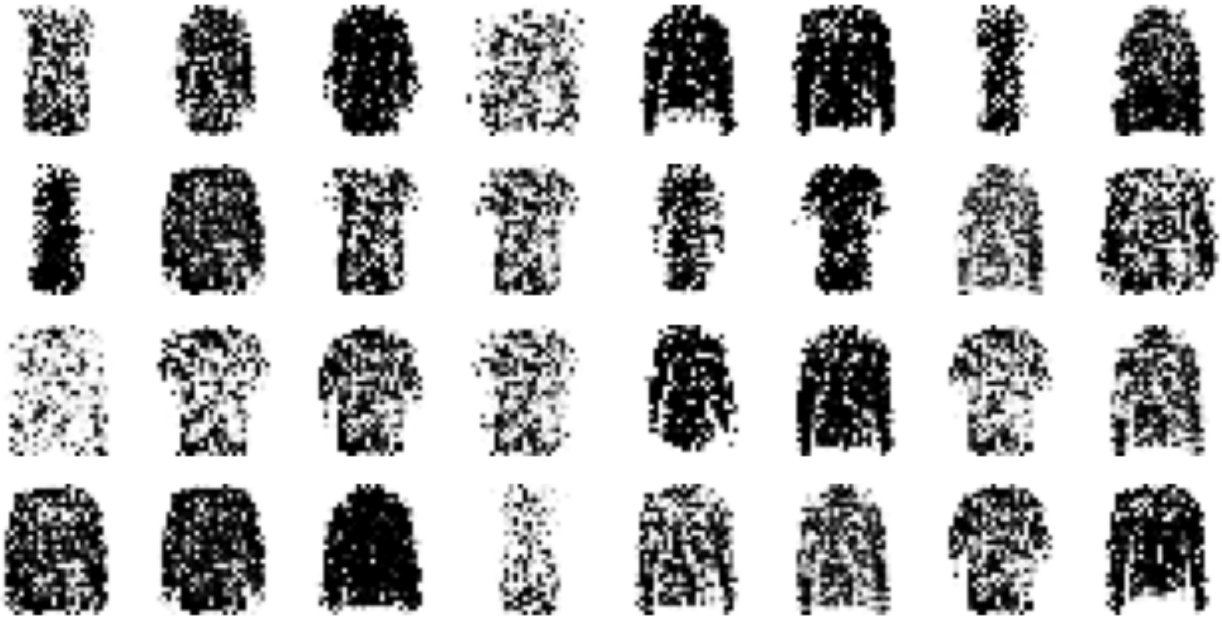
```
Epoch [2/5] ...
```



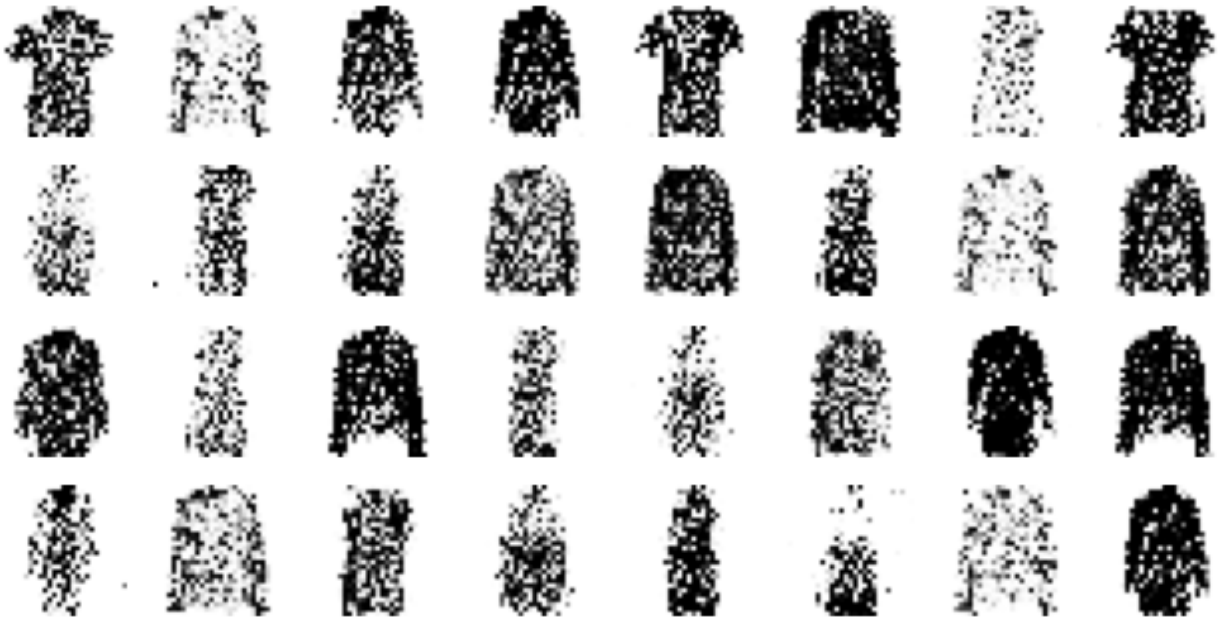
```
Epoch [3/5] ...
```



Epoch [4/5] ...



Epoch [5/5] ...



Deep Convolutional GAN

In [41]:
```
codings_size = 100

deep_generator = keras.models.Sequential([
    keras.layers.Dense(7 * 7 * 128, input_shape=[codings_size]),
    keras.layers.Reshape([7, 7, 128]),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2, padding="SAME",
                                 activation="selu"),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2, padding="SAME",
                                 activation="tanh"),
])
deep_discriminator = keras.models.Sequential([
    keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="SAME",
                        activation=keras.layers.LeakyReLU(0.2),
                        input_shape=[28, 28, 1]),
    keras.layers.Dropout(0.4),
    keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="SAME",
                        activation=keras.layers.LeakyReLU(0.2)),
    keras.layers.Dropout(0.4),
    keras.layers.Flatten(),
    keras.layers.Dense(1, activation="sigmoid")
])
deep_gan = keras.models.Sequential([deep_generator, deep_discriminator])
```

In [42]:
```
deep_discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
deep_discriminator.trainable = False
deep_gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

In [43]:
```
X_train_deep_gan = X_train.reshape(-1, 28, 28, 1) * 2. - 1. # reshape and rescal
```

In [44]:
```
# Cargar las imagenes en batches
batch_size = 32
dataset = tf.data.Dataset.from_tensor_slices(X_train_deep_gan)
dataset = dataset.shuffle(1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

In [45]:
```
# Entrenar el modelo Deep GAN con 5 iteraciones
train_gan(deep_gan, dataset, batch_size, codings_size, n_epochs=5)
```

```
Epoch [1/5] ...
```



```
Epoch [2/5] ...
```



```
Epoch [3/5] ...
```



Epoch [4/5] ...



Epoch [5/5] ...



StyleGAN

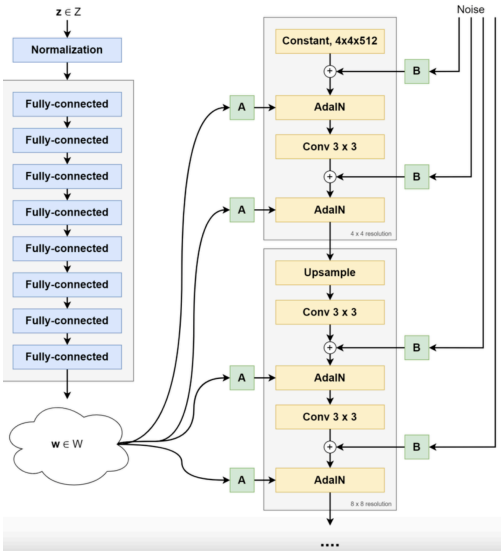
StyleGAN toma la idea de las GAN y la mejora para permitir un mayor control sobre la generación de imágenes.

1. "Mapa de estilo": En lugar de generar una imagen de manera directa, el ruido aleatorio primero pasa por una red llamada red de mapeo. Esto transforma el ruido en una representación intermedia que controla diferentes características del resultado final.

2. Control de estilos a diferentes escalas: Por ejemplo, los aspectos generales como la forma de la cara se controlan en las primeras capas, mientras que los detalles más finos, como las arrugas o el cabello, se controlan en las capas posteriores

3. Arquitectura de generación basada en "AdaIN" (Adaptive Instance Normalization): permite que el "estilo" (características visuales como los colores y las texturas) se ajuste en diferentes etapas del generador, lo que permite una gran flexibilidad.

Arquitectura de StyleGAN



In [1]:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

from collections import OrderedDict
import pickle

import numpy as np

import IPython
```

In [2]:

```
class MyLinear(nn.Module):
    """Linear layer with equalized learning rate and custom learning rate multip
    def __init__(self, input_size, output_size, gain=2**(0.5), use_wscale=False,
        super().__init__()
        he_std = gain * input_size**(-0.5) # He init
        # Equalized learning rate and custom learning rate multiplier.
        if use_wscale:
            init_std = 1.0 / lrmul
            self.w_mul = he_std * lrmul
        else:
            init_std = he_std / lrmul
            self.w_mul = lrmul
        self.weight = torch.nn.Parameter(torch.randn(output_size, input_size) *
        if bias:
            self.bias = torch.nn.Parameter(torch.zeros(output_size))
            self.b_mul = lrmul
        else:
            self.bias = None

    def forward(self, x):
        bias = self.bias
        if bias is not None:
            bias = bias * self.b_mul
        return F.linear(x, self.weight * self.w_mul, bias)
```

In [3]:

```
class MyConv2d(nn.Module):
    """Conv layer with equalized learning rate and custom learning rate multipli
    def __init__(self, input_channels, output_channels, kernel_size, gain=2**(0.
```

```
                 intermediate=None, upscale=False):
        super().__init__()
        if upscale:
            self.upscale = Upscale2d()
        else:
            self.upscale = None
        he_std = gain * (input_channels * kernel_size ** 2) ** (-0.5) # He init
        self.kernel_size = kernel_size
        if use_wscale:
            init_std = 1.0 / lrmul
            self.w_mul = he_std * lrmul
        else:
            init_std = he_std / lrmul
            self.w_mul = lrmul
        self.weight = torch.nn.Parameter(torch.randn(output_channels, input_char
        if bias:
            self.bias = torch.nn.Parameter(torch.zeros(output_channels))
            self.b_mul = lrmul
        else:
            self.bias = None
        self.intermediate = intermediate

    def forward(self, x):
        bias = self.bias
        if bias is not None:
            bias = bias * self.b_mul

        have_convolution = False
        if self.upscale is not None and min(x.shape[2:]) * 2 >= 128:
            # this is the fused upscale + conv from StyleGAN, sadly this seems i
            # this really needs to be cleaned up and go into the conv...
            w = self.weight * self.w_mul
            w = w.permute(1, 0, 2, 3)
            # probably applying a conv on w would be more efficient. also this q
            w = F.pad(w, (1,1,1,1))
            w = w[:, :, 1:, 1:]+ w[:, :, :-1, 1:] + w[:, :, 1:, :-1] + w[:, :, :
            x = F.conv_transpose2d(x, w, stride=2, padding=(w.size(-1)-1)//2)
            have_convolution = True
        elif self.upscale is not None:
            x = self.upscale(x)

        if not have_convolution and self.intermediate is None:
            return F.conv2d(x, self.weight * self.w_mul, bias, padding=self.kern
        elif not have_convolution:
            x = F.conv2d(x, self.weight * self.w_mul, None, padding=self.kernel_

        if self.intermediate is not None:
            x = self.intermediate(x)
        if bias is not None:
            x = x + bias.view(1, -1, 1, 1)
        return x
```

In [4]:

```
class NoiseLayer(nn.Module):
    """adds noise. noise is per pixel (constant over channels) with per-channel
    def __init__(self, channels):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(channels))
        self.noise = None
```

```
    def forward(self, x, noise=None):
        if noise is None and self.noise is None:
            noise = torch.randn(x.size(0), 1, x.size(2), x.size(3), device=x.dev
        elif noise is None:
            # here is a little trick: if you get all the noiselayers and set eac
            # modules .noise attribute, you can have pre-defined noise.
            # Very useful for analysis
            noise = self.noise
        x = x + self.weight.view(1, -1, 1, 1) * noise
        return x
```

In [5]:

```
class StyleMod(nn.Module):
    def __init__(self, latent_size, channels, use_wscale):
        super(StyleMod, self).__init__()
        self.lin = MyLinear(latent_size,
                            channels * 2,
                            gain=1.0, use_wscale=use_wscale)

    def forward(self, x, latent):
        style = self.lin(latent) # style => [batch_size, n_channels*2]
        shape = [-1, 2, x.size(1)] + (x.dim() - 2) * [1]
        style = style.view(shape)  # [batch_size, 2, n_channels, ...]
        x = x * (style[:, 0] + 1.) + style[:, 1]
        return x
```

In [6]:

```
class PixelNormLayer(nn.Module):
    def __init__(self, epsilon=1e-8):
        super().__init__()
        self.epsilon = epsilon
    def forward(self, x):
        return x * torch.rsqrt(torch.mean(x**2, dim=1, keepdim=True) + self.epsi
```

In [7]:

```
class BlurLayer(nn.Module):
    def __init__(self, kernel=[1, 2, 1], normalize=True, flip=False, stride=1):
        super(BlurLayer, self).__init__()
        kernel=[1, 2, 1]
        kernel = torch.tensor(kernel, dtype=torch.float32)
        kernel = kernel[:, None] * kernel[None, :]
        kernel = kernel[None, None]
        if normalize:
            kernel = kernel / kernel.sum()
        if flip:
            kernel = kernel[:, :, ::-1, ::-1]
        self.register_buffer('kernel', kernel)
        self.stride = stride

    def forward(self, x):
        # expand kernel channels
        kernel = self.kernel.expand(x.size(1), -1, -1, -1)
        x = F.conv2d(
            x,
            kernel,
            stride=self.stride,
            padding=int((self.kernel.size(2)-1)/2),
            groups=x.size(1)
        )
```

```
        return x

def upscale2d(x, factor=2, gain=1):
    assert x.dim() == 4
    if gain != 1:
        x = x * gain
    if factor != 1:
        shape = x.shape
        x = x.view(shape[0], shape[1], shape[2], 1, shape[3], 1).expand(-1, -1,
        x = x.contiguous().view(shape[0], shape[1], factor * shape[2], factor *
    return x

class Upscale2d(nn.Module):
    def __init__(self, factor=2, gain=1):
        super().__init__()
        assert isinstance(factor, int) and factor >= 1
        self.gain = gain
        self.factor = factor
    def forward(self, x):
        return upscale2d(x, factor=self.factor, gain=self.gain)
```

In [8]:

```
class G_mapping(nn.Sequential):
    def __init__(self, nonlinearity='lrelu', use_wscale=True):
        act, gain = {'relu': (torch.relu, np.sqrt(2)),
                     'lrelu': (nn.LeakyReLU(negative_slope=0.2), np.sqrt(2))}[nc
        layers = [
            ('pixel_norm', PixelNormLayer()),
            ('dense0', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense0_act', act),
            ('dense1', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense1_act', act),
            ('dense2', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense2_act', act),
            ('dense3', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense3_act', act),
            ('dense4', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense4_act', act),
            ('dense5', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense5_act', act),
            ('dense6', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense6_act', act),
            ('dense7', MyLinear(512, 512, gain=gain, lrmul=0.01, use_wscale=use_
            ('dense7_act', act)
        ]
        super().__init__(OrderedDict(layers))

    def forward(self, x):
        x = super().forward(x)
        # Broadcast
        x = x.unsqueeze(1).expand(-1, 18, -1)
        return x

class Truncation(nn.Module):
    def __init__(self, avg_latent, max_layer=8, threshold=0.7):
        super().__init__()
        self.max_layer = max_layer
        self.threshold = threshold
        self.register_buffer('avg_latent', avg_latent)
    def forward(self, x):
```

```
        assert x.dim() == 3
        interp = torch.lerp(self.avg_latent, x, self.threshold)
        do_trunc = (torch.arange(x.size(1)) < self.max_layer).view(1, -1, 1)
        return torch.where(do_trunc, interp, x)
```

In [9]:

```
class LayerEpilogue(nn.Module):
    """Things to do at the end of each layer."""
    def __init__(self, channels, dlatent_size, use_wscale, use_noise, use_pixel_
        super().__init__()
        layers = []
        if use_noise:
            layers.append(('noise', NoiseLayer(channels)))
        layers.append(('activation', activation_layer))
        if use_pixel_norm:
            layers.append(('pixel_norm', PixelNorm()))
        if use_instance_norm:
            layers.append(('instance_norm', nn.InstanceNorm2d(channels)))
        self.top_epi = nn.Sequential(OrderedDict(layers))
        if use_styles:
            self.style_mod = StyleMod(dlatent_size, channels, use_wscale=use_wsc
        else:
            self.style_mod = None
    def forward(self, x, dlatents_in_slice=None):
        x = self.top_epi(x)
        if self.style_mod is not None:
            x = self.style_mod(x, dlatents_in_slice)
        else:
            assert dlatents_in_slice is None
        return x


class InputBlock(nn.Module):
    def __init__(self, nf, dlatent_size, const_input_layer, gain, use_wscale, us
        super().__init__()
        self.const_input_layer = const_input_layer
        self.nf = nf
        if self.const_input_layer:
            # called 'const' in tf
            self.const = nn.Parameter(torch.ones(1, nf, 4, 4))
            self.bias = nn.Parameter(torch.ones(nf))
        else:
            self.dense = MyLinear(dlatent_size, nf*16, gain=gain/4, use_wscale=u
        self.epi1 = LayerEpilogue(nf, dlatent_size, use_wscale, use_noise, use_p
        self.conv = MyConv2d(nf, nf, 3, gain=gain, use_wscale=use_wscale)
        self.epi2 = LayerEpilogue(nf, dlatent_size, use_wscale, use_noise, use_p

    def forward(self, dlatents_in_range):
        batch_size = dlatents_in_range.size(0)
        if self.const_input_layer:
            x = self.const.expand(batch_size, -1, -1, -1)
            x = x + self.bias.view(1, -1, 1, 1)
        else:
            x = self.dense(dlatents_in_range[:, 0]).view(batch_size, self.nf, 4,
        x = self.epi1(x, dlatents_in_range[:, 0])
        x = self.conv(x)
        x = self.epi2(x, dlatents_in_range[:, 1])
        return x
```

```
class GSynthesisBlock(nn.Module):
    def __init__(self, in_channels, out_channels, blur_filter, dlatent_size, gai
        # 2**res x 2**res # res = 3..resolution_log2
        super().__init__()
        if blur_filter:
            blur = BlurLayer(blur_filter)
        else:
            blur = None
        self.conv0_up = MyConv2d(in_channels, out_channels, kernel_size=3, gain=
                                 intermediate=blur, upscale=True)
        self.epi1 = LayerEpilogue(out_channels, dlatent_size, use_wscale, use_nc
        self.conv1 = MyConv2d(out_channels, out_channels, kernel_size=3, gain=ga
        self.epi2 = LayerEpilogue(out_channels, dlatent_size, use_wscale, use_nc

    def forward(self, x, dlatents_in_range):
        x = self.conv0_up(x)
        x = self.epi1(x, dlatents_in_range[:, 0])
        x = self.conv1(x)
        x = self.epi2(x, dlatents_in_range[:, 1])
        return x
```

In [10]:

```
class G_synthesis(nn.Module):
    def __init__(self,
        dlatent_size        = 512,          # Disentangled latent (W) dimensiona
        num_channels        = 3,            # Number of output color channels.
        resolution          = 1024,         # Output resolution.
        fmap_base           = 8192,         # Overall multiplier for the number
        fmap_decay          = 1.0,          # log2 feature map reduction when do
        fmap_max            = 512,          # Maximum number of feature maps in
        use_styles          = True,         # Enable style inputs?
        const_input_layer   = True,         # First layer is a learned constant?
        use_noise           = True,         # Enable noise inputs?
        randomize_noise     = True,         # True = randomize noise inputs ever
        nonlinearity        = 'lrelu',      # Activation function: 'relu', 'lrel
        use_wscale          = True,         # Enable equalized learning rate?
        use_pixel_norm      = False,        # Enable pixelwise feature vector no
        use_instance_norm   = True,         # Enable instance normalization?
        dtype               = torch.float32,  # Data type to use for activations
        blur_filter         = [1,2,1],      # Low-pass filter to apply when resa
        ):

        super().__init__()
        def nf(stage):
            return min(int(fmap_base / (2.0 ** (stage * fmap_decay))), fmap_max)
        self.dlatent_size = dlatent_size
        resolution_log2 = int(np.log2(resolution))
        assert resolution == 2**resolution_log2 and resolution >= 4

        act, gain = {'relu': (torch.relu, np.sqrt(2)),
                     'lrelu': (nn.LeakyReLU(negative_slope=0.2), np.sqrt(2))}[nc
        num_layers = resolution_log2 * 2 - 2
        num_styles = num_layers if use_styles else 1
        torgbs = []
        blocks = []
        for res in range(2, resolution_log2 + 1):
            channels = nf(res-1)
            name = '{s}x{s}'.format(s=2**res)
            if res == 2:
                blocks.append((name,
```

```
                                InputBlock(channels, dlatent_size, const_input_la
                                       use_noise, use_pixel_norm, use_instance_no

            else:
                blocks.append((name,
                               GSynthesisBlock(last_channels, channels, blur_fil
                last_channels = channels
        self.torgb = MyConv2d(channels, num_channels, 1, gain=1, use_wscale=use_
        self.blocks = nn.ModuleDict(OrderedDict(blocks))

    def forward(self, dlatents_in):
        # Input: Disentangled latents (W) [minibatch, num_layers, dlatent_size].
        # lod_in = tf.cast(tf.get_variable('lod', initializer=np.float32(0), tra
        batch_size = dlatents_in.size(0)
        for i, m in enumerate(self.blocks.values()):
            if i == 0:
                x = m(dlatents_in[:, 2*i:2*i+2])
            else:
                x = m(x, dlatents_in[:, 2*i:2*i+2])
        rgb = self.torgb(x)
        return rgb
```

In [11]:

```
g_all = nn.Sequential(OrderedDict([
    ('g_mapping', G_mapping()),
    ('g_synthesis', G_synthesis())
]))
```

In [13]:

```
g_all.load_state_dict(torch.load('./karras2019stylegan-ffhq-1024x1024.for_g_all.
```

Out[13]:

```
<All keys matched successfully>
```

In [15]:

```
%matplotlib inline
from matplotlib import pyplot
import torchvision

def generate_faces(nb_rows = 2, nb_cols = 5):
    device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
    g_all.eval()
    g_all.to(device)

    torch.manual_seed(20)

    nb_samples = nb_rows * nb_cols
    latents = torch.randn(nb_samples, 512, device=device)
    with torch.no_grad():
        imgs = g_all(latents)
        imgs = (imgs.clamp(-1, 1) + 1) / 2.0 # normalization to 0..1 range
    imgs = imgs.cpu()

    imgs = torchvision.utils.make_grid(imgs, nrow=nb_cols)

    pyplot.figure(figsize=(15, 6))
    pyplot.imshow(imgs.permute(1, 2, 0).detach().numpy())
```

In [16]:

```
generate_faces(nb_rows = 2, nb_cols = 5)
```



Interpolación

El objetivo de la interpolación es generar una secuencia de vectores latentes que empiecen en z_1 y terminen en z_2, pasando por varios puntos intermedios de manera suave. Para esto, se utiliza la interpolación lineal.

$$z(t) = (1 - t) \cdot z_1 + t \cdot z_2$$

Entonces, si tomamos $t = 0, 0.1, 0.2, \ldots, 1$ para generar múltiples vectores latentes intermedios y cada uno de estos vectores intermedios se pasa por la StyleGAN para generar una imagen, da como resultado una secuencia de imágenes que se transforman suavemente entre la imagen generada por z_1 y z_2.

In [19]:

```
def interpolation(nb_latents = 25, nb_interp = 10):
    device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
    # Get a few Images
    fixed_latents = [torch.randn(1, 512, device=device) for _ in range(nb_interp
    latents = []
    for i in range(len(fixed_latents)-1):
        latents.append(fixed_latents[i] + (fixed_latents[i + 1] - fixed_latents[
    latents.append(fixed_latents[-1])
    latents = torch.cat(latents, dim=0)

    %matplotlib inline
    with torch.no_grad():
        for latent in latents:
            latent = latent.to(device)
            img = g_all(latent.unsqueeze(0))
            img = img.clamp_(-1, 1).add_(1).div_(2.0)
            img = img.detach().squeeze(0).cpu().permute(1, 2, 0).numpy()

            pyplot.imshow(img)
            IPython.display.clear_output(True)
            pyplot.show()
```

In [21]:

```
interpolation(nb_latents = 25, nb_interp = 10)
```

